

Tipi di dato

In una tabella per ciascuna colonna possiamo definire diversi tipi di dato (dominio):

- [Dati numerici: interi](#)
- [Tipi di numeri in virgola mobile](#)
- [String \(stringa\)](#)
- [Date , Time \(data e ora\)](#)
- [JSON](#)
- [Attributi per interi](#)
- [Attributi per i campi di tipo stringa](#)
- [Attributi universali](#)
- [Attributi/Indici](#)

Dati numerici: interi

Tipo	Intervallo di valori	Solo se positivi (UNSIGNED)
BIT(M)	Da 1 a 64	/
TINYINT (1 byte)	da -128 a +127	da 0 a 255
SMALLINT (2 byte)	da -32 768 a +32 767	da 0 a 65 535
MEDIUMINT (3 byte)	da -8 388 608 a +8 388 607	da 0 a 16 777 215
INT (4 byte)	da -2 147 483 648 a +2 147 483 647	da 0 a 4 294 967 295
BIGINT (8 byte)	da -9 223 372 036 854 775 808 a +9 223 372 036 854 775 807	da 0 a 18 446 744 073 709 550 615

E' importante precisare che se all'interno di un campo di tipo numerico si cerca di inserire un valore maggiore di quanto ammesso dal tipo prescelto, MySQL produrrà un errore (nella configurazione di default del server).

Tipi di numeri in virgola mobile

Tipo	Tipo (sintassi)1	Tipo (sintassi deprecata)2	Spazio
FLOAT	FLOAT[(p)], con p compreso tra 0 e 23	FLOAT, FLOAT(M,D)*	4 byte
DOUBLE	FLOAT[(p)], con p compreso tra 24 e 53	DOUBLE, DOUBLE(M,D)*	8 byte
DECIMAL	DECIMAL(M,D)		Dipende da M*

Nel caso di FLOAT, puoi specificare opzionalmente la precisione, usando una sintassi tipo FLOAT(p), dove "p" è la precisione in bit.

Se la precisione è tra 0 e 23, la colonna sarà considerata di tipo FLOAT e userà 4 byte.

Se la precisione è tra 24 e 53, sarà considerata di tipo DOUBLE e userà 8 byte.

MySQL permette anche una sintassi non standard, del tipo FLOAT(M,D) (o DOUBLE(M,D)).

M è il numero totale di cifre che puoi memorizzare (prima e dopo il punto decimale).

D è il numero di cifre dopo il punto decimale.

Per esempio, se dichiari una colonna come FLOAT(7,4), puoi memorizzare fino a 7 cifre in totale, con 4 cifre dopo il punto decimale.

Quindi i numeri vanno da -999.9999 a 999.9999.

1 La sintassi FLOAT(p) è supportata ma ignorata: non influisce realmente sul tipo

2 <https://dev.mysql.com/doc/refman/8.4/en/precision-math-decimal-characteristics.html>

Dati stringa

Tipo e lunghezza massima consentita

Tipo	Lunghezza massima
CHAR(n)	255 caratteri
VARCHAR(n)	65.535 byte(1)
BINARY(b)	255 byte
VARBINARY(b)	65.535 byte
TINYTEXT	255 caratteri
TINYBLOB	255 byte
TEXT	65.535 caratteri
BLOB	65.535 byte
MEDIUMTEXT	16.777.215 caratteri
MEDIUMBLOB	16.777.215 byte
LONGTEXT	4.294.967.295 caratteri
LOBLOB	4.294.967.295 byte
ENUM('value1','value2',...)	ENUM usa 1 o 2 byte a seconda del numero di valori
SET('value1','value2',...)	64 valori distinti

1 Il limite è 65.535 byte perché dipende dal tipo di codifica adottata.

Con *utf8mb4* (default in MySQL) il massimo è ~**16383** caratteri.

I tipi **CHAR** e **VARCHAR** sono sicuramente i tipi più utilizzati.

La differenza tra questi due tipi è data dal fatto che **CHAR(n)** ha *lunghezza fissa*, **VARCHAR(n)** ha *lunghezza variabile*.

Questo significa che in una colonna **CHAR(10)** tutti i valori memorizzati occuperanno lo spazio di 10 caratteri anche se verranno inseriti solo 3 caratteri.

I tipi **TEXT** e **BLOB** (Binary Large Object) consentono di memorizzare grandi quantità di dati:

- TEXT è utilizzato per dati di tipo testuale,
- BLOB è utilizzato per ospitare dati binary (ad esempio il sorgente di un'immagine)

BINARY e VARBINARY

I tipi **BINARY** e **VARBINARY** sono simili a CHAR e VARCHAR, tranne per il fatto che *memorizzano stringhe binarie* anziché stringhe non binarie: memorizzano stringhe di byte anziché stringhe di caratteri.

Per questi campi il *set di caratteri* e la *collation*, il confronto e l'ordinamento si basano sui valori numerici dei byte memorizzati.

TEXT vs VARCHAR()

TEXT

- dimensione massima fissa di 65535 caratteri (non è possibile limitare la dimensione massima)
- prende 2 + c byte di spazio su disco, dove c è la lunghezza della stringa memorizzata.
- indice: può essere indicizzato solo con un prefix index.

VARCHAR (M)

- dimensione massima variabile di byte M
- M deve essere compreso tra 1 e 65535
- prende 1 + c byte (per $M \leq 255$) o 2 + c (per $256 \leq M \leq 65535$) byte di spazio su disco dove c è la lunghezza della stringa memorizzata
- può essere parte di un indice

Se è necessario memorizzare stringhe più lunghe di circa 64 KB, utilizzare MEDIUMTEXT o LONGTEXT. VARCHAR non supporta la memorizzazione di valori così grandi.

Tipi ENUM e SET

I tipi ENUM e SET sono un tipo di dato di testo in cui le colonne possono avere solo dei valori predefiniti.

ENUM: Tipo di dato ENUMerazione.

Contiene un insieme di valori prefissati tra cui scegliere: si può inserire solamente uno dei valori previsti.

I valori sono inseriti tra parentesi (elenco separato da virgola) dopo la dichiarazione ENUM.

genere **ENUM('F', 'M', 'NB')**

La colonna genere accetterà solamente i valori F, M o NB. Se proviamo a mettere un valore diverso con il comando INSERT, MYSQL restituirà errore.

SET: è una estensione di ENUM.

```
interessi SET('a','b','c','d')
```

Come per ENUM i valori sono fissi e disposti dopo la dichiarazione SET; tuttavia, le colonne SET possono assumere più di un valore tra quelli previsti.

DateTime

Tali tipi di dati sono molto utili quando si ha a che fare con informazioni riguardanti la data e l'orario.

Di seguito una tabella riepilogativa

Tipo	Formato	Intervallo
DATETIME	YYYY-MM-DD HH:MM:SS	'1000-01-01 00:00:00' a '9999-12-31 23:59:59'
DATE	YYYY-MM-DD	'1000-01-01' a '9999-12-31'
TIME	HH:MM:SS	'-838:59:59' a '838:59:59'
YEAR	YYYY	un anno compreso fra 1901 e 2155, oppure 0000
TIMESTAMP	YYYY-MM-DD HH:MM:SS	'1970-01-01 00:00:01' UTC a '2038-01-19 03:14:07' UTC

I campi di tipo DATETIME contengono sia la data che l'orario.

I valori all'interno di questi campi possono essere inseriti sia sotto forma di stringhe che di numeri.

Sia **DATETIME** sia **TIMESTAMP** possono memorizzare in automatico la data.

Per ottenere ciò in fase di definizione del campo bisogna impostare il valore di default di memorizzazione (es):

```
ins TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
```

```
data DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
```

DATETIME o TIMESTAMP?

TIMESTAMP

Il tipo TIMESTAMP memorizza data e ora utilizzando il fuso orario del server per le conversioni automatiche da e verso UTC.

Quando memorizzi un valore in una colonna TIMESTAMP, MySQL lo converte automaticamente nel fuso orario UTC (Coordinated Universal Time) e lo memorizza in quel formato. Quando lo leggi, il valore viene riconvertito nel fuso orario del server o in quello del client (se configurato).

Questo rende il TIMESTAMP utile quando devi mostrare gli stessi dati di data e ora a utenti in fusi orari diversi, poiché MySQL gestisce automaticamente la conversione da e verso UTC.

TIMESTAMP copre solo il range di date da 1970-01-01 00:00:01 UTC fino a 2038-01-19 03:14:07 UTC.

Se hai bisogno di memorizzare date al di fuori di questo intervallo, non è adatto.

DATETIME

Il tipo DATETIME memorizza la data e l'ora così come sono, senza conversioni di fuso orario. Il valore viene memorizzato esattamente come viene inserito, senza alcuna considerazione del fuso orario del server o del client.

Questo significa che DATETIME è utile quando vuoi che la data e l'ora restino fisse, indipendentemente dal fuso orario dell'utente o del server.

È una buona scelta se i dati di data e ora devono essere gli stessi per tutti, ovunque si trovino.

DATETIME ha un range molto più ampio, da 1000-01-01 00:00:00 a 9999-12-31 23:59:59, quindi può essere usato per date molto più lontane nel passato o nel futuro rispetto a TIMESTAMP.

JSON

MySQL supporta JSON nativo come tipo di dati per gli oggetti nella notazione JSON.

Rende facile l'archiviazione, l'interrogazione e il recupero di documenti di tipo JSON piuttosto che archivarli come stringhe di testo o BLOB binari (vedi MariaDB).

Per fare ciò mette a disposizione una serie di funzioni⁽¹⁾

Sintassi per la definizione di un attributo di tipo JSON

`columnName JSON`

(1) <https://dev.mysql.com/doc/refman/8.0/en/json-function-reference.html>

Attributi

Per i campi numerici

Si tratta di un'istruzione ulteriore che viene passata al DBMS in fase di creazione (o modifica) di una colonna di tabella.

- **AUTO_INCREMENT** - Aumenta automaticamente il valore di una colonna aggiungendo 1 al valore massimo già registrato nella tabella. È opportuno utilizzarlo in combinazione con NOT NULL. Può essere applicato a tutti i tipi numerici interi.
- **UNSIGNED**(1) - un campo contrassegnato con UNSIGNED non accetta valori negativi.
- **ZEROFILL**(2) - viene utilizzato per memorizzare i numeri con degli zeri davanti nel caso in cui la lunghezza sia inferiore a quella massima prevista. Se per esempio si dichiara un campo INT ZEROFILL e si desidera memorizzare il valore "1234", esso verrà visualizzato come "0000001234" con M(11) - sempre UNSIGNED.

1 A partire da MySQL 8.0.17 è deprecato per FLOAT, DOUBLE e DECIMAL;

2 A partire da MySQL 8.0.17 è deprecato; vedi documentazione mysql

AUTO_INCREMENT (sequence)

In MySQL, una sequenza è un elenco di interi generati nell'ordine crescente, vale a dire 1,2,3...

Impostare l'attributo **AUTO_INCREMENT** in una colonna, tipicamente una colonna chiave primaria, crea automaticamente una sequenza in MySQL.

Le seguenti regole vengono applicate quando si utilizza l'attributo **AUTO_INCREMENT**:

- Ciascuna tabella ha solo una colonna **AUTO_INCREMENT** il cui tipo di dati è *intero*.
- La colonna **AUTO_INCREMENT** deve essere indicizzata, il che significa che può essere:
 - **PRIMARY KEY** o **UNIQUE**
- La colonna **AUTO_INCREMENT** deve avere un vincolo **NOT NULL**.

*Quando si imposta l'attributo **AUTO_INCREMENT** in una colonna, MySQL aggiunge automaticamente il vincolo **NOT NULL** alla colonna implicitamente.*

Attributi per i campi di tipo stringa

BINARY

- L'unico vincolo/opzione che può essere utilizzato per i campi destinati ad ospitare dati stringa è **BINARY** il quale può essere utilizzato con **CHAR(n)** o **VARCHAR(n)** qualora questi campi siano destinati ad ospitare dati binari (pur non rendendosi necessario utilizzare un campo della famiglia BLOB)
- **CHAR(n) BINARY** e **VARCHAR(n) BINARY** differiscono dal tipo di dato **BINARY** e **VARBINARY** per la codifica e la collation utilizzata(1)

1 <https://dev.mysql.com/doc/refman/8.0/en/binary-varbinary.html>

Attributi universali

Possono essere utilizzati sia con campi numerici sia con campi di tipo stringa.

- **DEFAULT** - Può essere utilizzato con tutti i tipi di dati ad eccezione di TEXT e BLOB. Serve per indicare un valore di default per il campo qualora questo non venga compilato.
- **NULL / NOT NULL** - Può essere utilizzato con tutti i tipi di campi e serve per definire se un dato campo può avere un valore NULL oppure no.
- **COMMENT** - Aggiunge un commento descrittivo sull'uso del campo.

Vincoli

- **CHECK (expression)** - consente di imporre un vincolo al dato da inserire.
- **FOREIGN KEY** - consente di imporre un vincolo riferito alla chiave esterna.

Attributi/Indici

- **UNIQUE** - Con UNIQUE si imposta una regola di unicità, questo significa che nessun dato contenuto nella colonna può essere ripetuto: ogni dato deve, quindi, essere unico e se si cerca di inserire un dato duplicato si riceve un errore. Può essere nullo.
- **PRIMARY KEY** - Può essere utilizzato con tutti i tipi di dati (numerici e stringa) ed è una sorta di variante di UNIQUE che consente di creare un indice primario sulla tabella (campo chiave).

- **INDEX** o **KEY** (colonne) - E' utilizzato per creare un'indice nella tabella ai fini di migliorare le performances di accesso ai dati.

Cenni sull'uso degli indici

introduzione al concetto, verranno trattati in modo approfondito in altra sezione

- Servono ad ottimizzare le performance del database.
- Un indice è una struttura dati ausiliaria che consente di recuperare più velocemente i dati di una tabella, evitandone la lettura dell'intero contenuto (full table scan), tramite una selezione più mirata.
- devono essere usati consapevolmente per non ottenere l'effetto contrario ovvero rallentare il db.